

2025 데이터 크리에이터 캠프

**DATA
CREATOR
CAMP**

대학부 팀명 : 동데입구



과학기술정보통신부

NIA 지능정보원
한국지능정보사회진흥원



목차

1. 데이터 이해

2. 데이터 처리

3. 데이터 증강

4. 모델 설계

5. 학습 단계

6. 추가 사항

7. 수행 결과



Mission 1: 오염원 첫 발견



1. 데이터 이해

✓ 목표

- 위성 이미지에서 굴뚝 위치를 탐지하는 Object Detection 모델 개발

✓ 데이터 소스

- Kompsat-3/3A 위성 이미지 데이터

✓ 데이터 구성

- 학습 데이터 : 이미지 8,052개 (TS_KS 폴더)
- 검증 데이터 : 이미지 1,006개 (VS_KS 폴더)
- 라벨링 : Bounding Box 좌표가 담긴 JSON 파일
- 클래스 : 단일 클래스 ('chimney')



1. 데이터 이해



이미지 파일 예시

```
{"K3_CHN_20130308050237_30.jpg59649":{"filename":"K3_CHN_20130308050237_30.jpg","size":59649,"regions":[{"shape_attributes":{"name":"rect","x":139,"y":327,"width":49,"height":103},"region_attributes":{"chi_id":"1"}}],"file_attributes":{"img_id":"K3_CHN_20130308050237_30","img_height":"512","img_width":"512","img_resolution":"0.7","img_time":"13030805","img_provided":"한국항공우주연구원","provided_nm":"한국지능정보사회진흥원"}}}}
```

JSON 파일 예시



2. 데이터 처리

❖ 제공된 JSON 형식의 라벨 데이터를 YOLOv8 모델 학습에 맞게 변환하는 전처리 진행

✓ 라벨 형식 변환:

각 이미지의 JSON 라벨 파일을 YOLO가 요구하는 .txt 형식으로 변환

✓ 좌표 정규화:

JSON 파일에 있던 x, y, width, height 픽셀 좌표를 이미지의 전체 너비와 높이로 나누어 **0과 1 사이의 값으로 정규화**. 또한, 좌측 상단 기준의 (x, y) 좌표를 YOLO 형식에 맞는 **바운딩 박스의 중심 좌표 (center_x, center_y)**로 변환



2. 데이터 처리

❖ 제공된 JSON 형식의 라벨 데이터를 YOLOv8 모델 학습에 맞게 변환하는 전처리 진행

✓ 데이터셋 구조화

YOLOv8 학습을 위해 images와 labels 폴더를 각각 train과 val로 나누어 구성하고, 이 구조를 정의한 chimney_dataset.yaml 파일을 생성하여 모델이 데이터를 쉽게 찾을 수 있도록 함

✓ 데이터 무결성 확인

전처리 과정에서 학습 데이터셋의 이미지 1개가 누락된 것을 발견하고, 해당 라벨을 학습에서 제외하여 오류를 방지

✓ 효율성 및 안정성 확보

다양한 이미지 확장자(.jpg, .png 등)를 모두 인식하여 파일 누락을 방지하고, 이미 변환 작업이 완료된 파일은 자동으로 건너뛰는(skip) 기능이 포함되어 있어 코드를 여러 번 실행해도 중복 작업 없이 효율적으로 처리



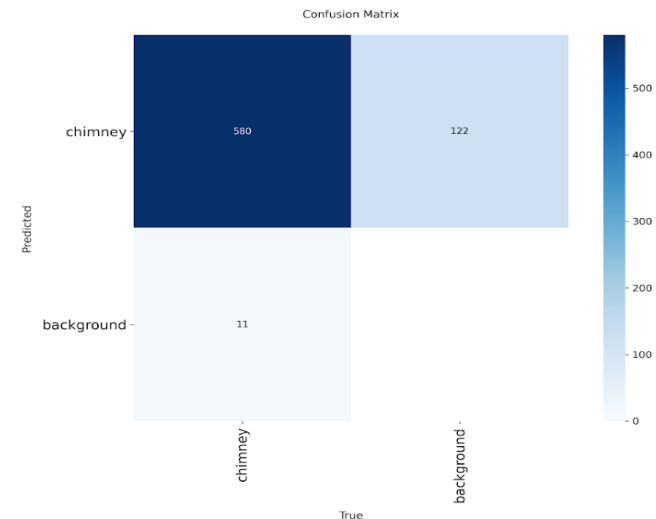
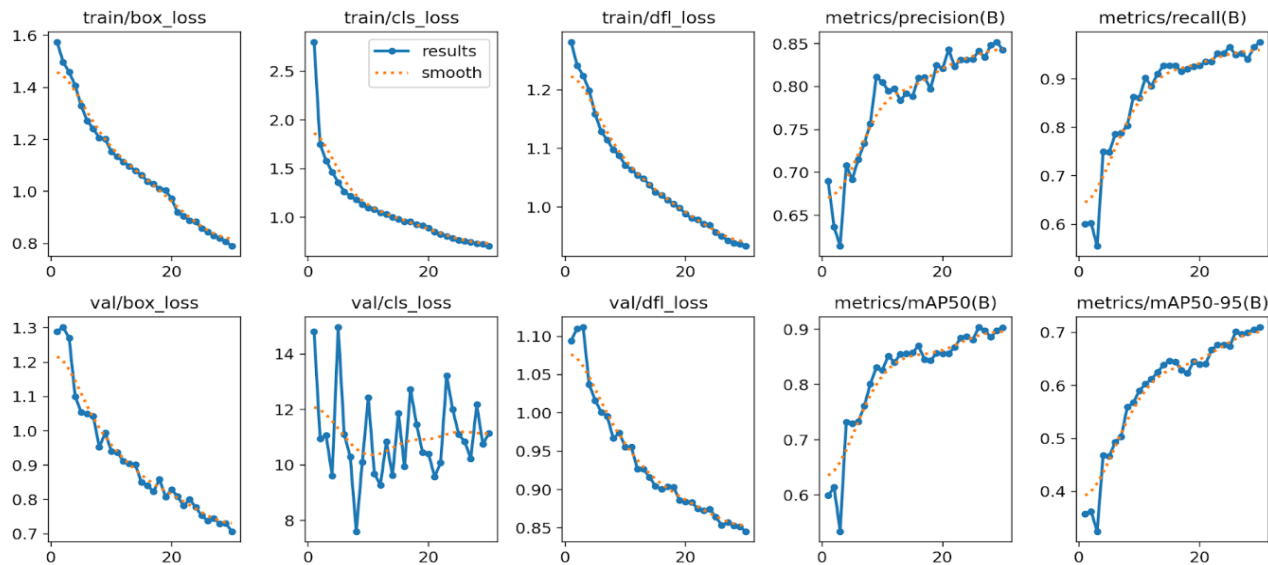
3. 데이터 증강

✓ 데이터 증강 미적용

모델의 탐지 정확도 및 손실 정도가 기대에 미치지 못하는 문제 발생

➔ 원인 : 원본 데이터에만 익숙해져 새로운 위성 이미지에 대해 일반화 성능 저하

❖ 데이터 증강 미적용 : 그래프 및 혼동행렬의 결과





3. 데이터 증강

✓ 데이터 증강 적용 (모델에 적합한 데이터 증강을 선별하여 적용)

1. 기하학적 변환

degrees = 10.0

-> 회전: 이미지를 ± 10 도 범위 내에서 무작위로 회전

translate = 0.1

-> 이동: 이미지를 상하좌우로 $\pm 10\%$ 범위 내에서 이동

scale = 0.1

-> 확대/축소: 이미지 크기를 $\pm 10\%$ 범위 내에서 확대/축소

shear = 5.0

-> 기울이기: 이미지를 ± 5.0 도 범위 내에서 전단 변환(기울이기)

flipud = 0.5

-> 50% 확률로 이미지를 상하로 뒤집음

※ 위성 이미지는 방향성이 상대적으로 중요하지 않으므로 효과적인 증강 기법으로 작용함
(위성 이미지에 매우 효과적)

2. 모자이크 및 믹스업

mosaic = 1.0

-> 100% 확률로 4개의 이미지를 하나로 합쳐 학습 데이터로 사용

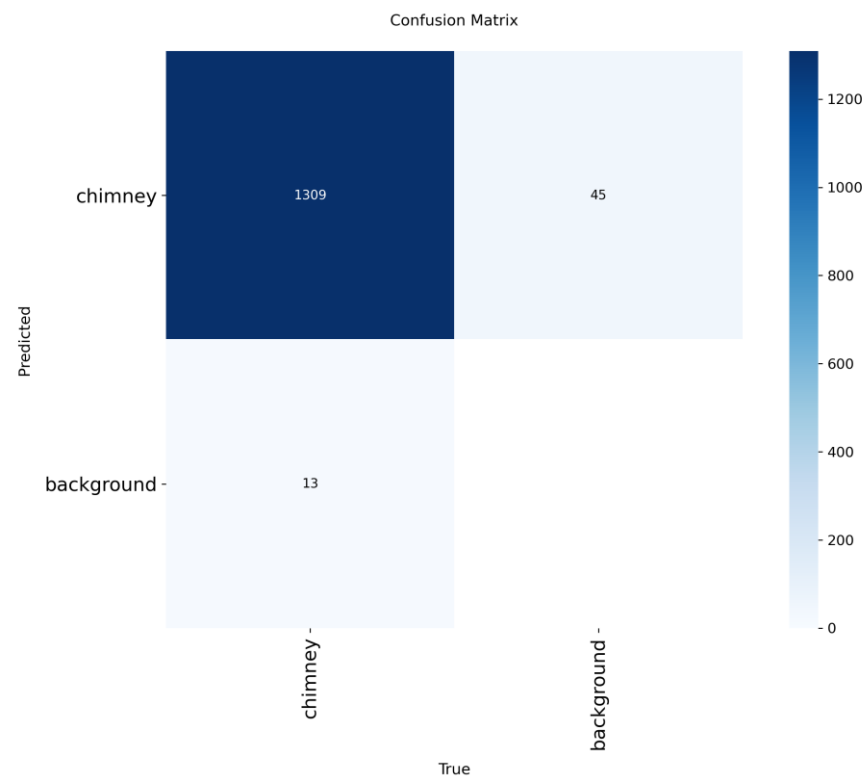
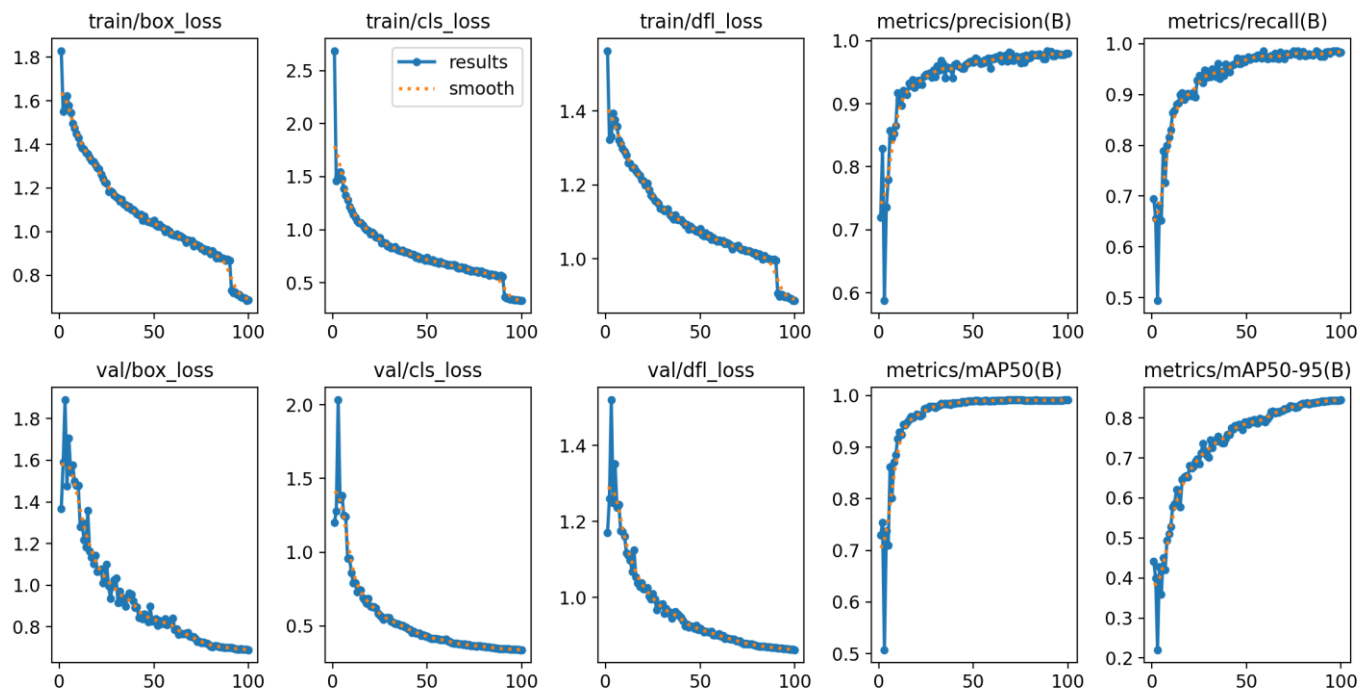
mixup = 0.1

-> 10% 확률로 두 이미지를 픽셀 단위로 섞어 라벨도 함께 조합



3. 데이터 증강

❖ 초기 데이터 증강 적용 : 그래프 및 혼동행렬 결과





3. 데이터 증강

✓ 데이터 증강 적용 (이전 모델의 성능을 개선하기 위해 데이터 증강의 수치를 조정)

1. scale: 확대/축소 범위 증가 (0.1 → 0.5)

- 초기 설정($\pm 10\%$)은 너무 변화가 적어 다양한 크기의 굴뚝을 학습하기에 부족함
- 위성 이미지는 촬영 고도나 해상도에 따라 굴뚝의 크기가 매우 다를 수 있음
- 스케일 범위를 $\pm 50\%$ 로 넓히면서, 모델은 **훨씬 더 다양한 크기의 굴뚝 이미지로 학습**할 수 있게 되었음
- 이로 인해 아주 작거나 큰 굴뚝에 대해서도 놓치지 않고 탐지할 수 있는 **강건함이 크게 향상**되었음

2. shear: 기울이기 강도 감소 (5.0 → 2.0)

- 5.0도의 전단 변환(shear)은 이미지를 **비현실적일 정도로 심하게 왜곡**시킬 수 있음
- 이런 과도한 변형은 오히려 굴뚝의 고유한 형태적 특징을 해치는 '노이즈'로 작용하여 모델의 학습을 방해한다고 판단함
- 기울이는 강도를 2.0도로 줄임으로써, **현실에서 관측 가능한 수준의 자연스러운 원근 왜곡**을 데이터에 추가함
- 이로 인해 모델이 불필요한 왜곡 대신 굴뚝의 핵심 특징에 더 집중하여 학습할 수 있게 되었음



3. 데이터 증강

✓ 데이터 증강 적용 (이전 모델의 성능을 개선하기 위해 데이터 증강의 수치를 조정)

3. flipud: 상하 반전 확률 감소 (0.5 → 0.3)

- 위성 이미지에서 상하 반전은 매우 효과적인 증강 기법이지만, 50%의 확률은 다소 과할 수 있음
- 모델이 학습하는 데이터의 절반이 뒤집힌 이미지라면, 오히려 원본 이미지의 고유한 그림자 방향이나 주변 환경과의 맥락을 학습하는 데 혼란을 줄 수 있음
- 30%로 낮춤으로써 **상하 방향의 불변성**이라는 이점은 계속 유지하면서도, 원본 이미지의 특징을 학습할 기회를 **더 많이** 제공하여 균형을 맞춤

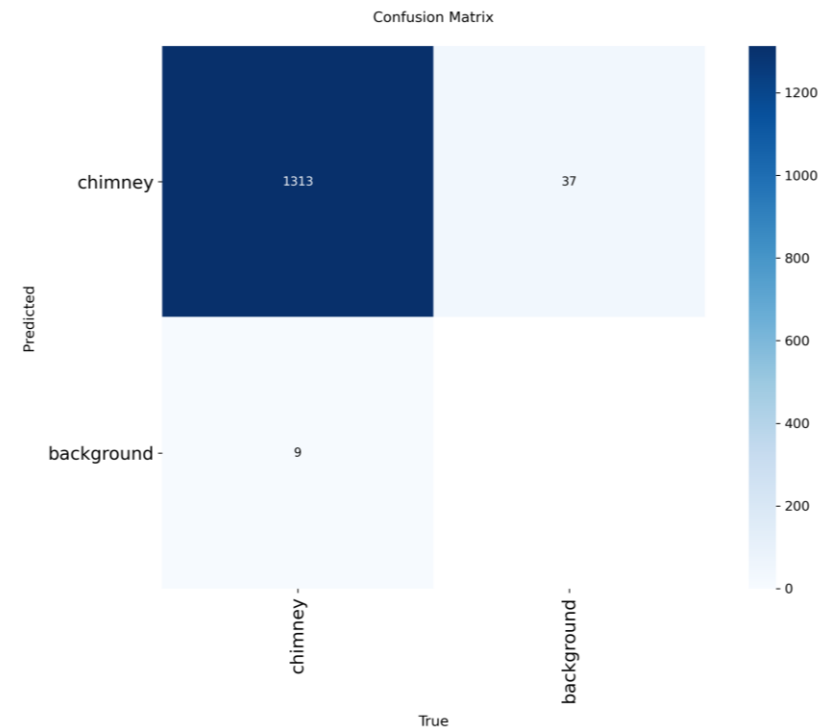
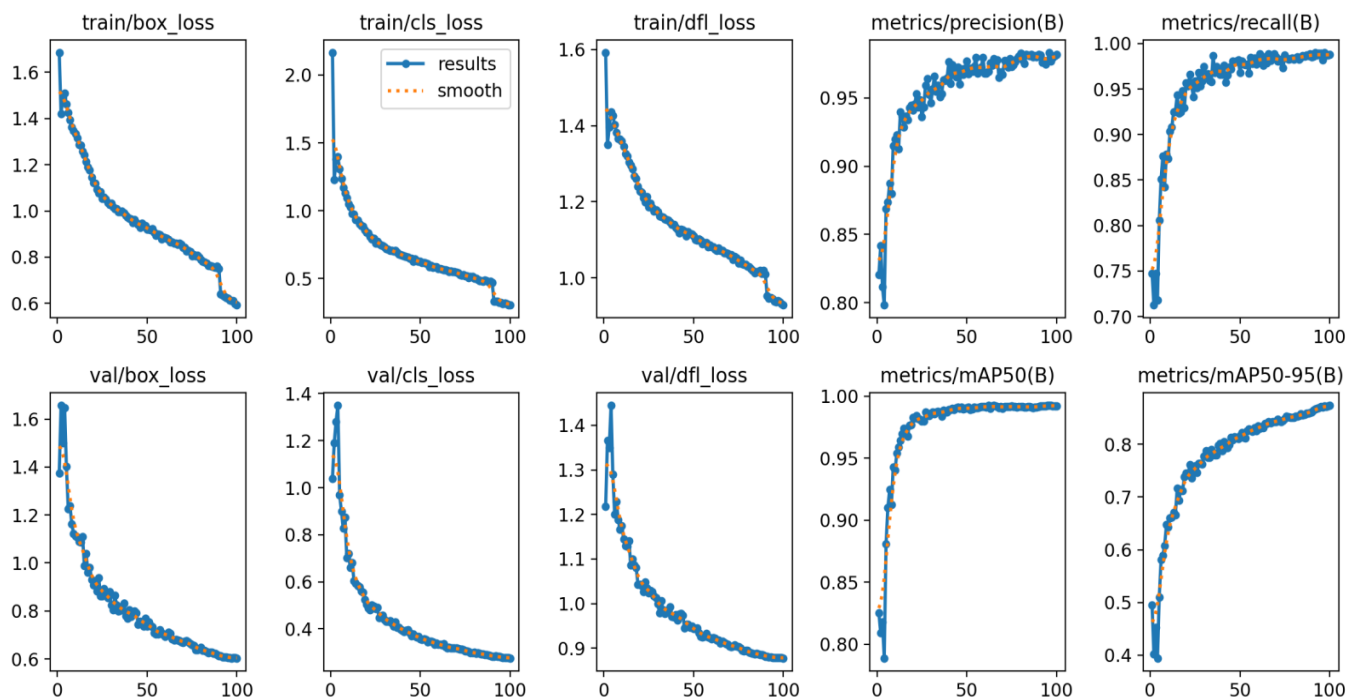
4. mosaic: 모자이크 적용 확률 감소 (1.0 → 0.8)

- 모자이크는 작은 객체 탐지에 매우 강력하지만, 100% 적용될 경우 모델이 학습 내내 온전한 한 장의 이미지를 보지 못하는 문제가 발생함
- 모델은 80%는 모자이크로 작은 객체와 다양한 배경 조합을 학습하고, 나머지 20%는 원본 전체 이미지를 보며 객체의 전체적인 맥락과 위치를 학습함



3. 데이터 증강

❖ 개선된 데이터 증강 적용 : 그래프 및 혼동행렬의 결과





4. 모델 설계

❖ 빠른 속도의 YOLOv8s 모델 -> 더 복잡한 특징 학습 & 정확도를 높이기 위한 YOLOv8m 채택

✓ 더 높은 특징 학습 능력

➤ 배경과 객체 구분:

위성 이미지 속 산업 단지에는 굴뚝 외에도 환풍구, 파이프, 기둥 등 비슷하게 생긴 구조물이 많음. s모델은 이런 구조물들을 굴뚝으로 혼동할 수 있지만, 학습 능력이 더 뛰어난 m모델은 굴뚝 특유의 그림자, 꼭대기 모양, 연기 흔적 등 더 세밀한 차이점을 구분하여 정확도를 높임

✓ 다양한 데이터에 대한 일반화 성능

➤ 크기와 형태 변화에 강인:

m모델은 더 많은 데이터를 처리하고 기억할 수 있어, 작게 나온 굴뚝, 일부가 가려진 굴뚝, 비스듬히 보이는 굴뚝 등 다양한 변수에 더 잘 대응할 수 있음. 즉, 더 많은 예외 케이스를 학습하여 어떤 상황에서도 굴뚝을 안정적으로 찾아내는 일반화 성능이 뛰어남



4. 모델 설계

❖ 빠른 속도의 YOLOv8s 모델 -> 더 복잡한 특징 학습 & 정확도를 높이기 위한 YOLOv8m 채택

✓ 정확도와 신뢰성 간의 trade-off

➤ 정확도 우선 선택:

초기 s 모델은 빠른 속도로 가능성을 확인하는 데 좋았지만, 최종 목표인 '높은 탐지 정확도'를 달성하기에는 부족했음. 따라서 약간의 학습 시간 증가를 감수하더라도, **결과의 신뢰도를 극대화할 수 있는 m 모델을 선택**하는 것이 합리적인 결정이었음. 프로젝트가 보유한 8,000장 이상의 충분한 데이터와 데이터 증강 기법이 m 모델의 강력한 학습 능력을 뒷받침함



5. 학습 단계

❖ 최적의 성능을 내기 위해 다음과 같은 하이퍼파라미터를 설정하여 학습을 진행

- **Imgsz = 1024**: 입력 이미지의 크기를 1024x1024으로 조정하여 학습
- **Batch = 24**: 한 번의 반복에서 24개의 이미지를 사용하여 모델을 업데이트
- **Epoch = 100**: 전체 데이터셋을 총 100번 반복하여 학습
- **Patience = 20**: **Early Stopping**을 위한 파라미터로, 20 epoch 동안 검증 성능이 개선되지 않으면 학습을 조기 종료하도록 설정함. 이는 불필요한 학습을 막고 과적합을 방지하며 최적의 모델을 찾는 데 도움
- **Seed = 42**: 결과의 재현성을 위해 랜덤 시드를 42로 고정
- **Workers = 8**: 8개의 CPU 코어를 데이터 로딩에 사용하여 GPU의 대기시간을 최소화하고 학습 효율을 높임



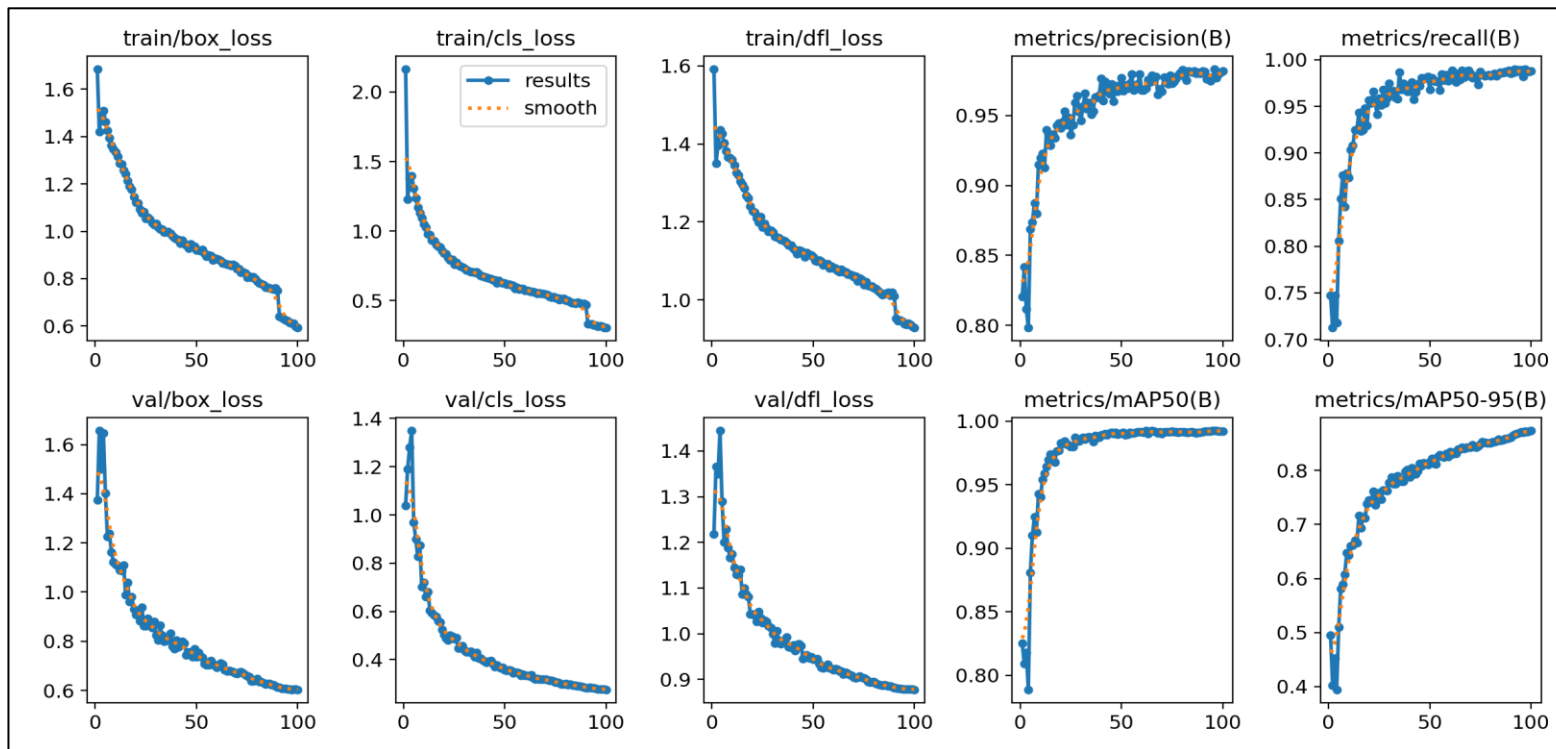
6. 추가 사항

- GPU : NVIDIA A100 GPU를 사용하여 대규모 이미지 데이터를 빠르고 효율적으로 학습
- 자동 하이퍼파라미터 최적화 : optimizer = 'auto' 로 설정하여 Ultralytics 프레임워크가 데이터셋 특성에 맞는 최적의 Optimizer와 Learning rate를 자동으로 찾아 적용하도록 했음



7. 수행결과

❖ 최종 성능 그래프



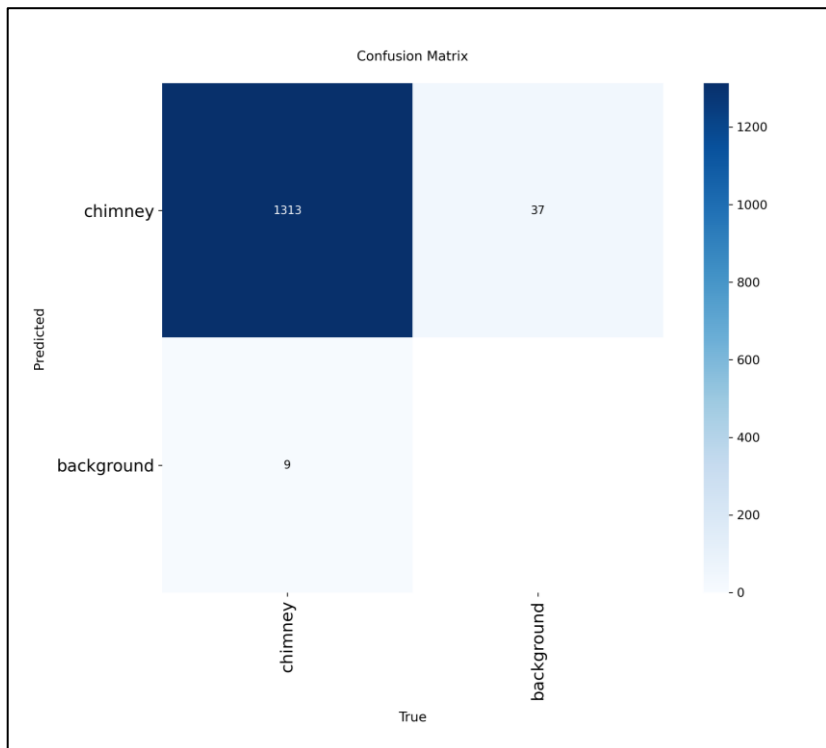
❖ mAP50: 0.992

❖ mAP50-95: 0.874



7. 수행결과

❖ 혼동행렬

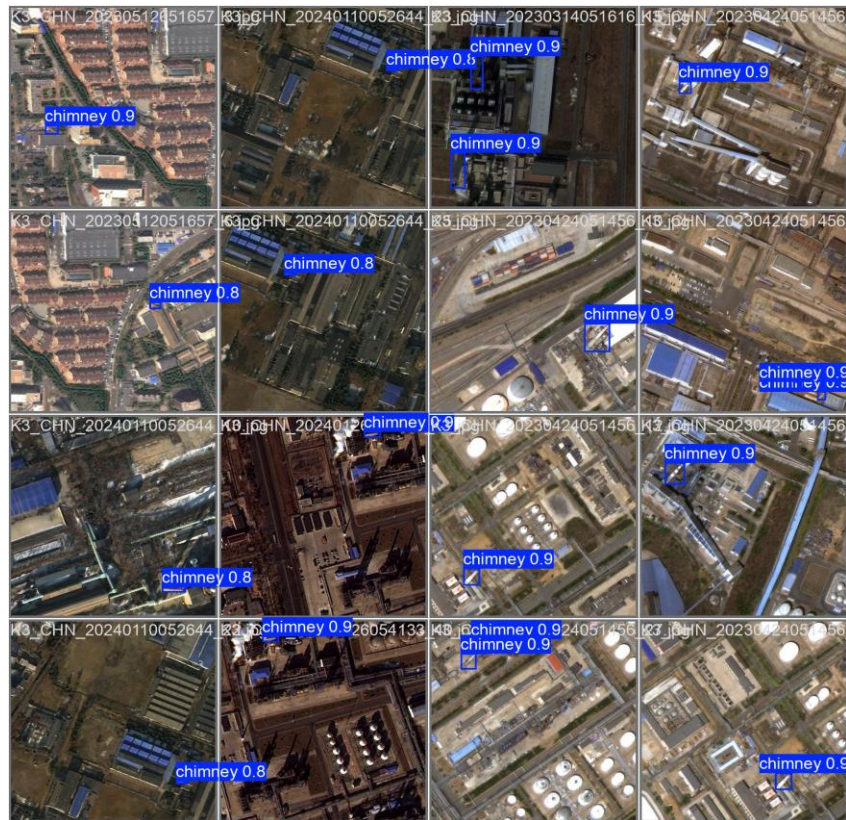


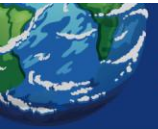
- 이미지 개수가 아니라 객체 단위로 집계되므로 validation 데이터 개수랑 confusion matrix 개수 총 합계랑 다른 것이 정상
 - validation에 1,006장 이미지가 있지만, 그 안에 굴뚝이 총 1321개 있어서 confusion matrix는 1321개 객체 기준으로 계산됨
- 단일 클래스에서도 FP(굴뚝 아닌데 굴뚝이라고 착각), FN(굴뚝인데 굴뚝 아니라고 착각)에 대한 오차가 cls_loss로 반영



7. 수행결과

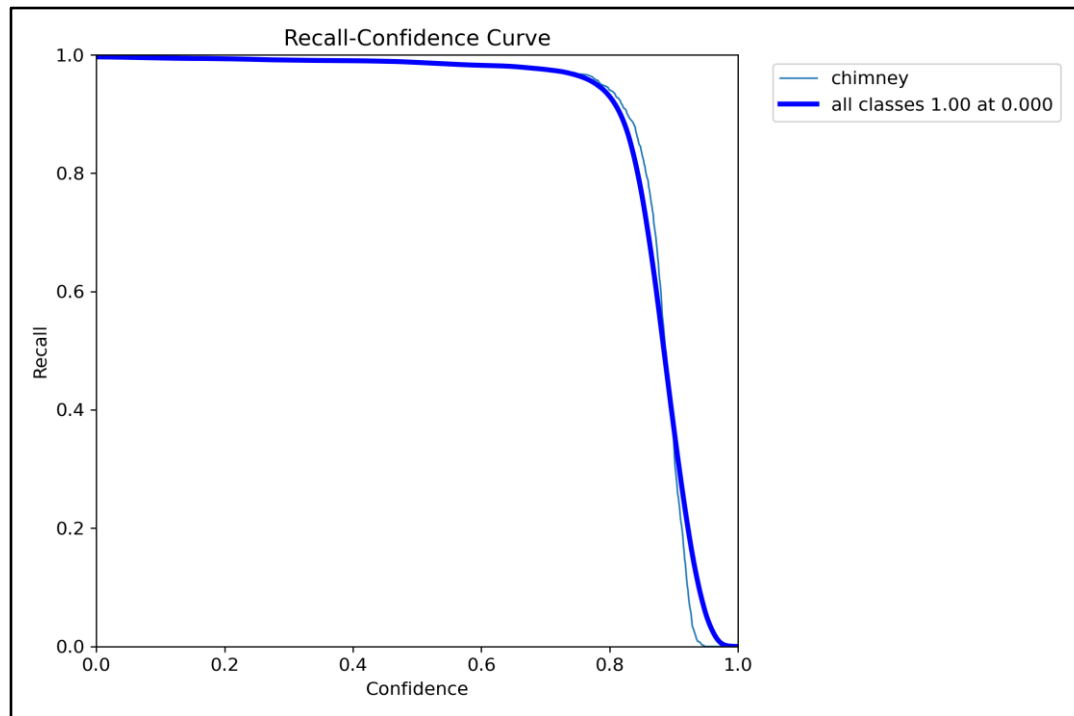
❖ 예측 결과 시각화



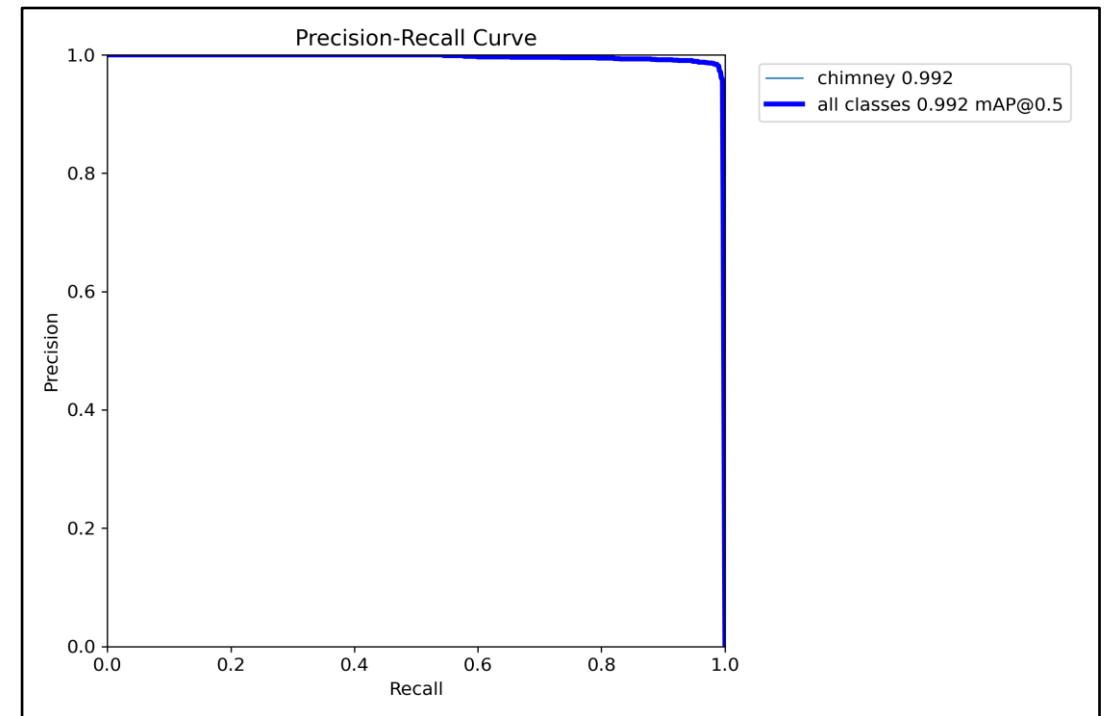


7. 수행결과

❖ Recall-Confidence Curve



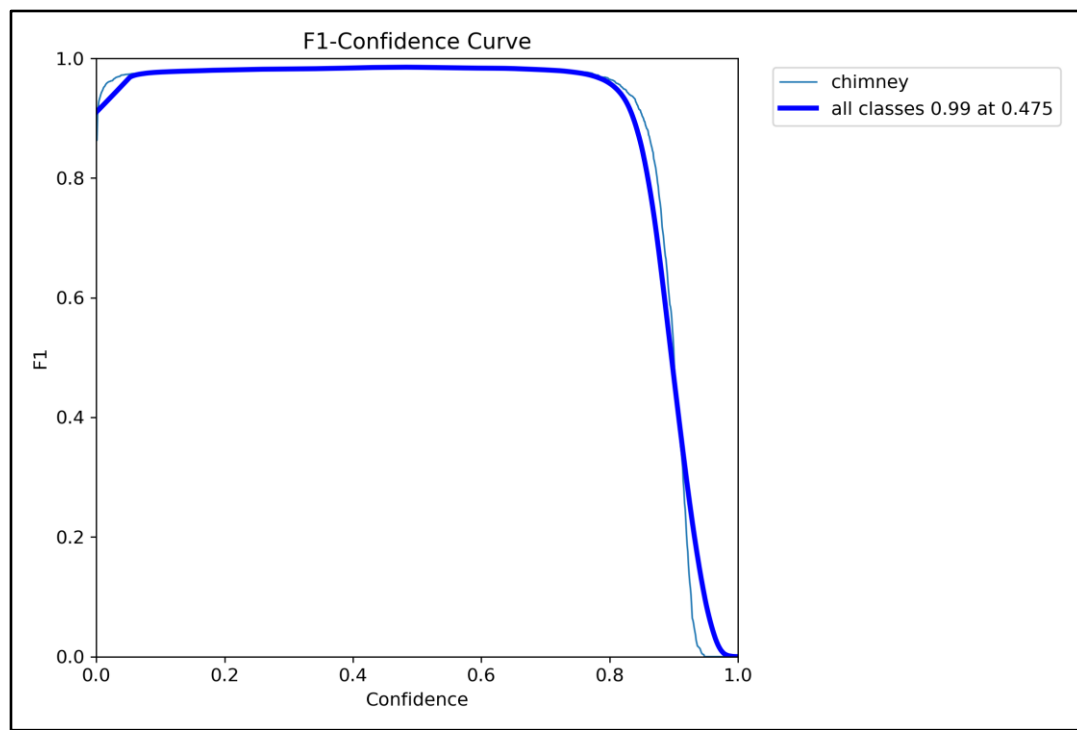
❖ Precision-Recall Curve



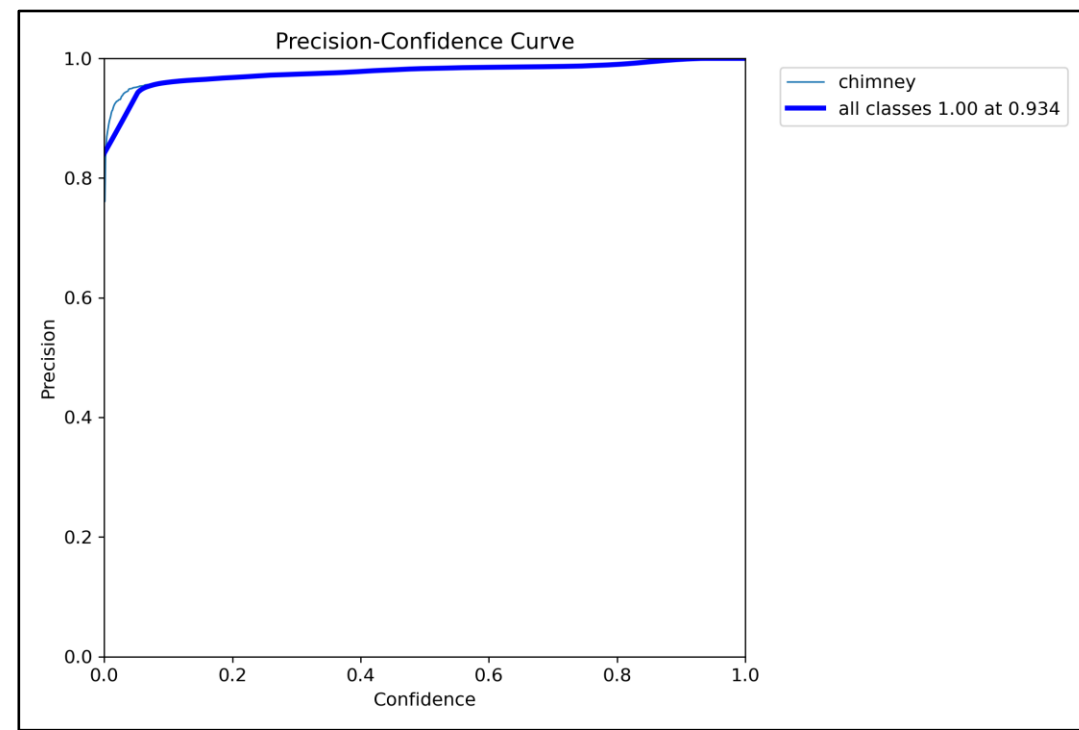


7. 수행결과

❖ F1-Confidence Curve



❖ Precision-Confidence Curve





Mission 2: 배출 규모 가늠하기



1. 데이터 이해

➤ 목표

- 위성 이미지로부터 굴뚝의 높이를 미터(m) 단위로 예측하는 Regression 모델 개발

➤ 데이터 소스

- Kompsat-3/3A 위성 이미지 데이터

➤ 데이터 구성

- 학습 데이터 : 이미지 8,052개에서 추출된 굴뚝 인스턴스 10,590개 (TS_KS 폴더)
- 검증 데이터 : 이미지 1,006개에서 추출된 굴뚝 인스턴스 1,323개 (VS_KS 폴더)
- 라벨링 : 굴뚝의 높이 값이 포함된 JSON 파일 (TL_KS_LINE, VL_KS_LINE 폴더)



1. 데이터 이해



이미지 파일 예시

```
{"K3_CHN_20130308050237_8.jpg70172":{"filename":"K3_CHN_20130308050237_8.jpg","size":70172,"regions":[{"shape_attributes":{"name":"polyline","all_points_x":[342,376],"all_points_y":[287,380]},"region_attributes":{"chi_id":"1","chi_height_m":"187.28"}}],"file_attributes":{"img_id":"K3_CHN_20130308050237_8","img_height":"512","img_width":"512","img_roll_tilt":"4.6120984245","img_pitch_tilt":"19.8605007474","img_resolution":"0.7","img_time":"13030805","img_provided":"한국항공우주연구원","provided_nm":"한국지능정보사회진흥원"}}}
```

JSON 파일 예시



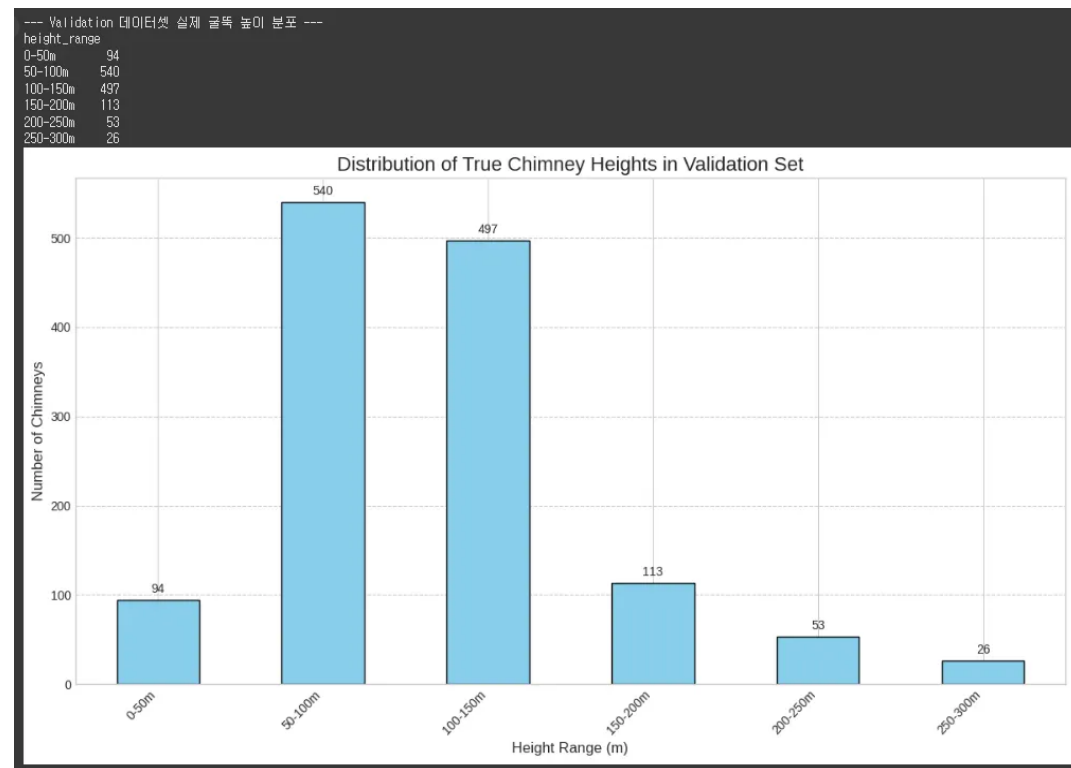
1. 데이터 이해

- 실제 Validation 굴뚝 크기 분포

- 0-50m : 94개
- 50-100m : 540개
- 100-150m : 497개
- 150-200m : 113개
- 200-250m : 53개
- 250-300m : 26개

- 평가 지표

- RMSE (Root Mean Square Error)를 기반으로 모델의 예측 오차를 평가





2. 데이터 처리

✓ 데이터 파싱 및 크롭:

JSON 파일에 포함된 line 좌표를 이용해 각 굴뚝의 위치에 **Bounding Box**를 생성함. 이후, 전체 위성 이미지에서 이 바운딩 박스 영역만 잘라내어(crop) 개별 굴뚝 이미지를 만듦. 이는 모델이 배경의 노이즈 없이 굴뚝 자체의 시각적 특징에만 집중하도록 하는 핵심 과정임

✓ 데이터셋 구성:

전처리된 크롭 이미지 경로와 해당 이미지의 높이 값을 DataFrame 형태로 정리하여 학습에 사용하기 용이한 구조를 만듦



2. 데이터 처리

✓ 바운딩 박스 패딩:

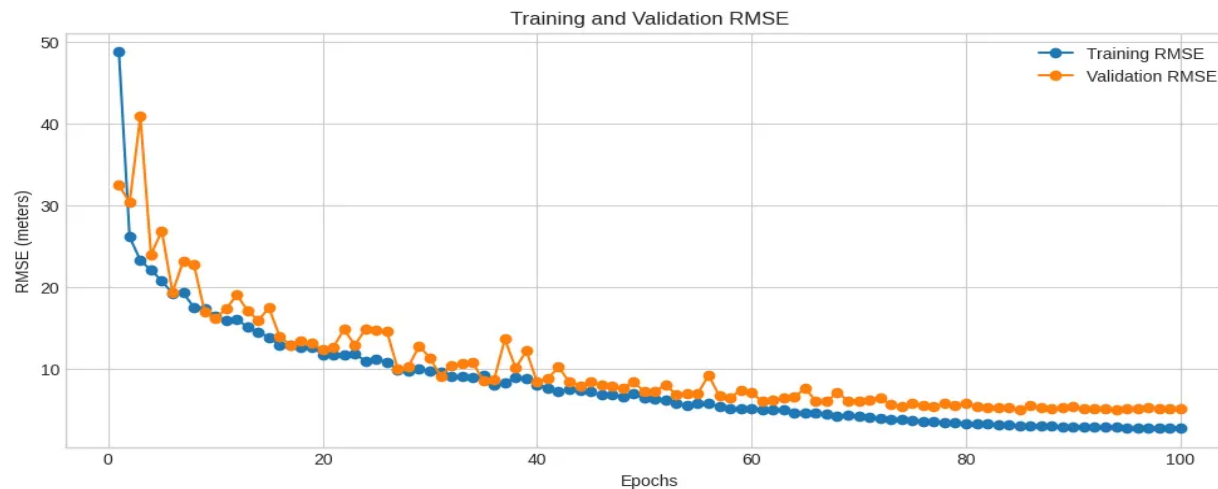
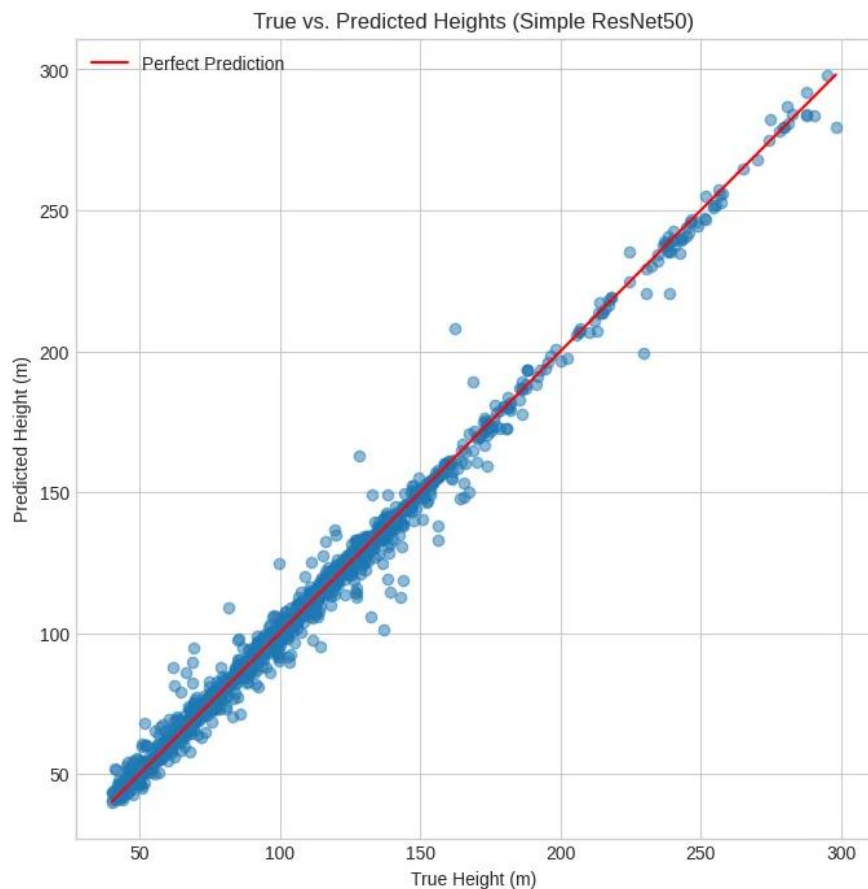
라벨링된 선 좌표의 최솟값과 최댓값을 이용해 바운딩 박스를 계산함. 하지만 이렇게 딱 맞는 박스를 사용하는 대신, 그 주위에 작은 여백, 즉 padding을 추가함

- **맥락적 단서 포함:** 너무 딱 맞는 박스는 굴뚝 가장자리의 중요한 시각적 정보를 잘라낼 수 있음. 지붕 위의 굴뚝 바닥이나 그림자 같은 주변 배경을 약간 포함함으로써, 모델은 더 많은 **맥락 정보**를 얻게 됨
- **가장자리 특징 손실 방지:** 이미지가 신경망에 의해 처리될 때, 맨 가장자리의 정보는 때때로 손실되거나 덜 표현될 수 있음. 패딩은 전체 굴뚝이 프레임 안에 명확하게 들어오도록 보장하여, 가장자리의 핵심 특징들이 우연히 잘려나가는 것을 방지함



3. 데이터 증강

❖ 3-1. ResNet-50 모델을 기반으로 모든 데이터에 동일한 데이터 증강 기법을 적용하여 실행한 결과



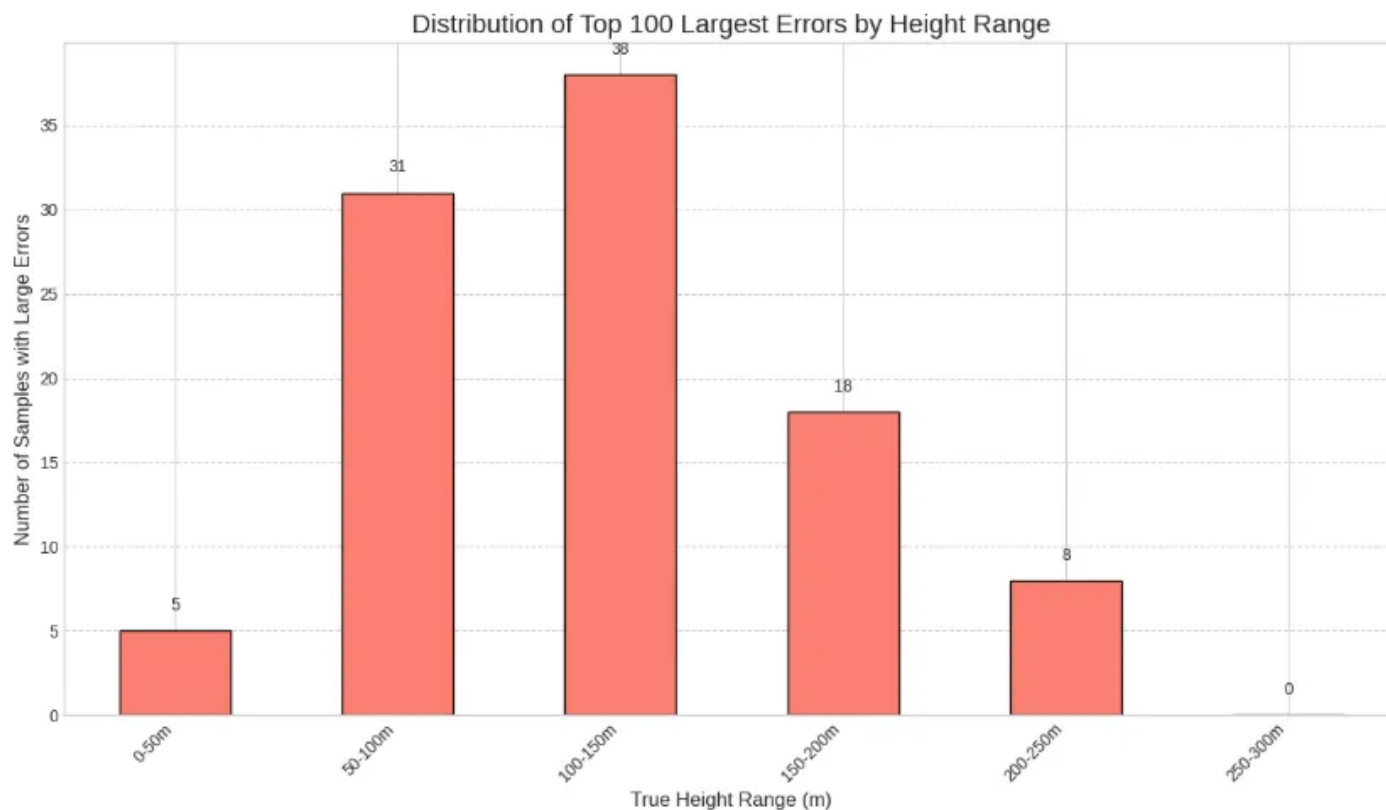
- Epoch : 100
- Val RMSE : 5.0764

✓ 충분히 좋은 RMSE지만 성능 개선을 위해 고민



3. 데이터 증강

❖ 3-2. 기존 모델이 특히 잘 못 맞춘 구간 시각화 (Top 100)



- 0-50m : 5개
- 50-100m : 31개
- 100-150m : 38개
- 150-200m : 18개
- 200-250m : 8개
- 250-300m : 0개



3. 데이터 증강

❖ 3-2. 기존 모델이 특히 잘 못 맞춘 구간 시각화 (Top 100)

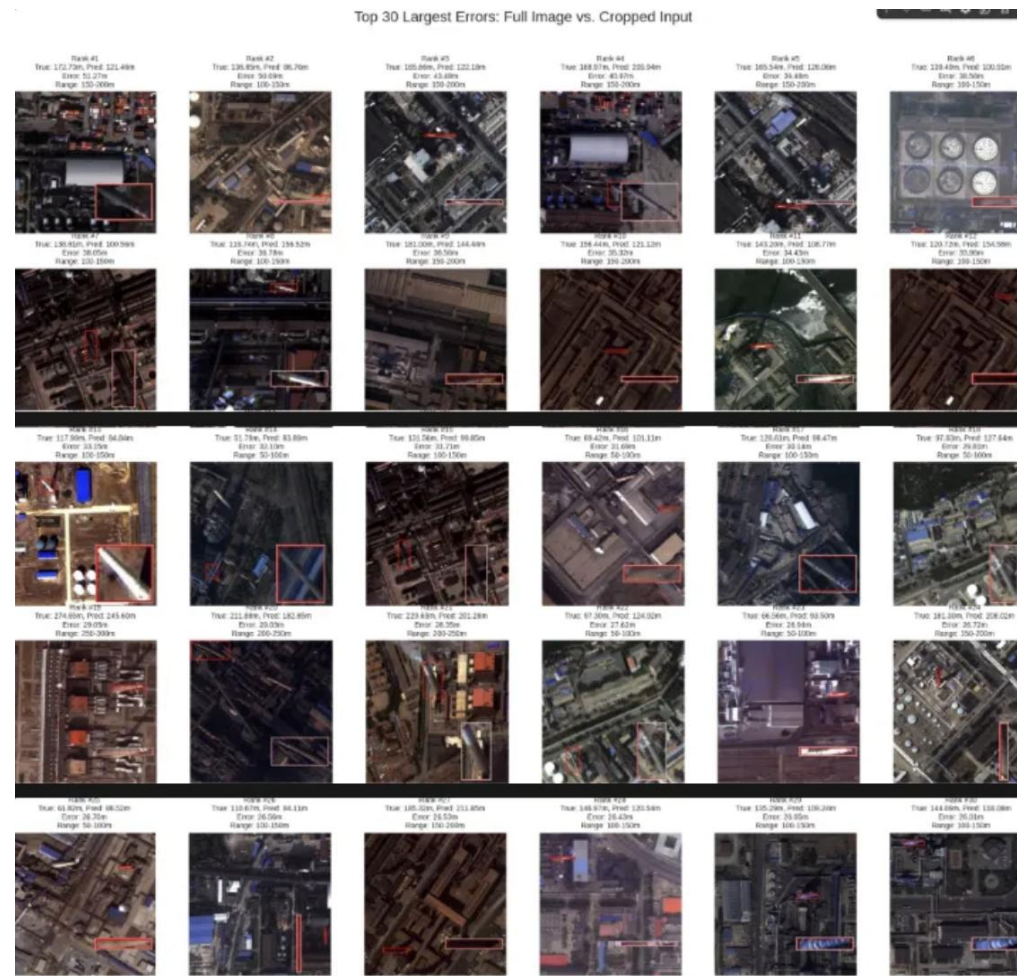
TOP 100 SAMPLES WITH LARGEST PREDICTION ERRORS					
	Rank	Height Range	True Height	Predicted Height	Absolute Error
171	1	150-200m	168.97	214.04	45.07
768	2	150-200m	162.47	197.51	35.04
1212	3	50-100m	61.82	94.94	33.12
1249	4	100-150m	138.61	106.89	31.72
560	5	100-150m	143.20	111.86	31.34
314	6	100-150m	132.25	101.78	30.47
254	7	50-100m	69.67	99.91	30.24
1024	8	100-150m	139.49	111.07	28.42
611	9	50-100m	84.95	113.25	28.30
1188	10	50-100m	69.12	97.15	28.03
956	11	150-200m	156.44	129.14	27.30
109	12	150-200m	186.45	160.26	26.19
777	13	200-250m	229.63	203.83	25.80
1215	14	100-150m	133.10	157.62	24.52
845	15	100-150m	144.09	119.63	24.46
315	16	100-150m	114.36	90.10	24.26
386	17	50-100m	88.31	64.39	23.92
122	18	100-150m	136.56	112.84	23.72
284	19	100-150m	115.33	138.92	23.59
1148	20	100-150m	120.15	143.39	23.24

가장 못 맞춘 구간 상위 10개만 보아도
50-200m 구간이 가장 많이 보이는 것을
확인할 수 있음



3. 데이터 증강

❖ 3-3. Top 30 Largest Error를 시각화





3. 데이터 증강

❖ 3-4. 해당 오차의 핵심 원인 및 경향 분석

- ✓ **Bbox의 부정확성 및 과도한 주변부 포함 (가장 핵심적인 문제)**
 - **경향:** Bbox가 굴뚝 자체보다 훨씬 크게 설정되어, 주변 건물, 바닥, 다른 구조물들이 이미지의 상당 부분을 차지함
 - **원인 분석:** ResNet과 같은 CNN 모델은 이미지 전체의 시각적 특징을 학습하여 높이를 추정. 입력 이미지에 굴뚝이 아닌 거대한 건물이 함께 포함되면, 모델은 굴뚝이 아닌 건물의 특징(크기, 복잡성)에 현혹되어 높이를 과대평가할 가능성이 있음
- ✓ **굴뚝의 비정형성 및 모호함**
 - **경향:** 길고 가는 파이프나 건물의 일부처럼 보이는 수평적 구조물들이 다수 포함되어 있음
 - **원인 분석:** 모델은 주로 수직적이고 명확한 형태의 굴뚝 이미지를 학습했을 가능성이 있음. 이처럼 가로로 길거나 형태가 모호한 객체의 '높이'를 추정하는 것은 매우 어려운 문제임. 모델이 '길이'를 '높이'로 오인하여 예측했을 수 있음



3. 데이터 증강

❖ 3-4. 해당 오차의 핵심 원인 및 경향 분석

✓ 강력한 그림자 및 낮은 대비

- **경향:** 굴뚝 본체보다 그림자가 훨씬 더 선명하고 크게 나타나거나, 주변 배경과 굴뚝의 색상 대비가 낮아 경계가 불분명함
- **원인 분석:** 그림자는 굴뚝의 2차원적 특징을 왜곡하여 모델이 실제 높이를 추정하는데 혼란을 줌. 그림자의 길이나 방향에 따라 예측값이 크게 흔들릴 수 있음. 낮은 대비의 이미지는 모델이 굴뚝의 정확한 경계와 질감을 파악하는 것을 어렵게 만들

✓ 주변 구조물의 복잡성

- **경향:** 파이프라인, 철골 구조물 등이 굴뚝 주변에 밀집해 있어 시각적으로 혼란스러움
- **원인 분석:** 특히 굴뚝과 유사한 선형적, 수직적 특징을 가진 다른 구조물들이 주변에 많을 경우 모델은 어떤 게 진짜 예측 대상인지 구분하는 데 어려움을 겪음. 이로 인해 주변부의 특징까지 함께 학습하여 오차가 발생함

가장 못 맞춘 세 구간(50~100m, 100~150m, 150~200m)에 대해 2가지 방법으로 나눠 접근한 내용을 3.5에서 설명



3. 데이터 증강

❖ 3-5. 방법1) 가중 샘플링의 적용

✓ 계기: 데이터 분석을 통한 문제점 발견

- 예측 오차가 무작위로 발생하는 것이 아니라 **50-200m** 라는 특정 높이 구간에 집중되어 있다는 패턴을 발견함

✓ 이유: "오답 노트 집중 공략" 전략

- 해결 전략:** 일반적인 무작위 학습 방식으로는 모델이 이 약점을 충분히 보완하기 어려움. 따라서 모델이 학습할 데이터를 선택할 때 **50-200m** 구간의 "**어려운 문제**"를 더 높은 확률로 뽑도록 학습 환경을 재설계하기로 결정함
- 선택한 기법: WeightedRandomSampler(가중 샘플링)** 는 이 전략을 가장 효과적이고 효율적으로 구현할 수 있는 방법임. 데이터셋이나 증강 코드를 복잡하게 수정할 필요 없이, DataLoader에 sampler 옵션 하나만 추가하여 '선별적 집중 학습'을 가능하게 함

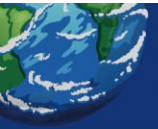


3. 데이터 증강

❖ 3-5. 방법1) 가중 샘플링의 적용

✓ 기대효과

- **약점 집중 보완:** 모델이 50-200m 구간의 데이터에 더 많이 노출되면서, 해당 구간에 대한 예측 정확도가 직접적으로 향상될 것임
- **전체 성능 개선:** 특정 구간의 큰 오차들이 줄어들면서, 전반적인 Validation RMSE가 개선될 것으로 기대됨



3. 데이터 증강

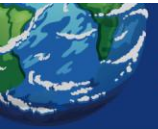
❖ 3-5. 방법1) 가중 샘플링의 적용 및 코드

❖ 가중 샘플링 기법 코드

```
#가중 샘플링(Weighted Sampling)
weights = []
for height in train_df_filtered['height']:
    if 50 <= height <= 200:
        weights.append(1.5) # 문제 구간에 1.5배 높은 가중치 부여
    else:
        weights.append(1.0) # 나머지 구간은 기본 가중치
sampler = WeightedRandomSampler(weights, num_samples=len(weights), replacement=True)
```

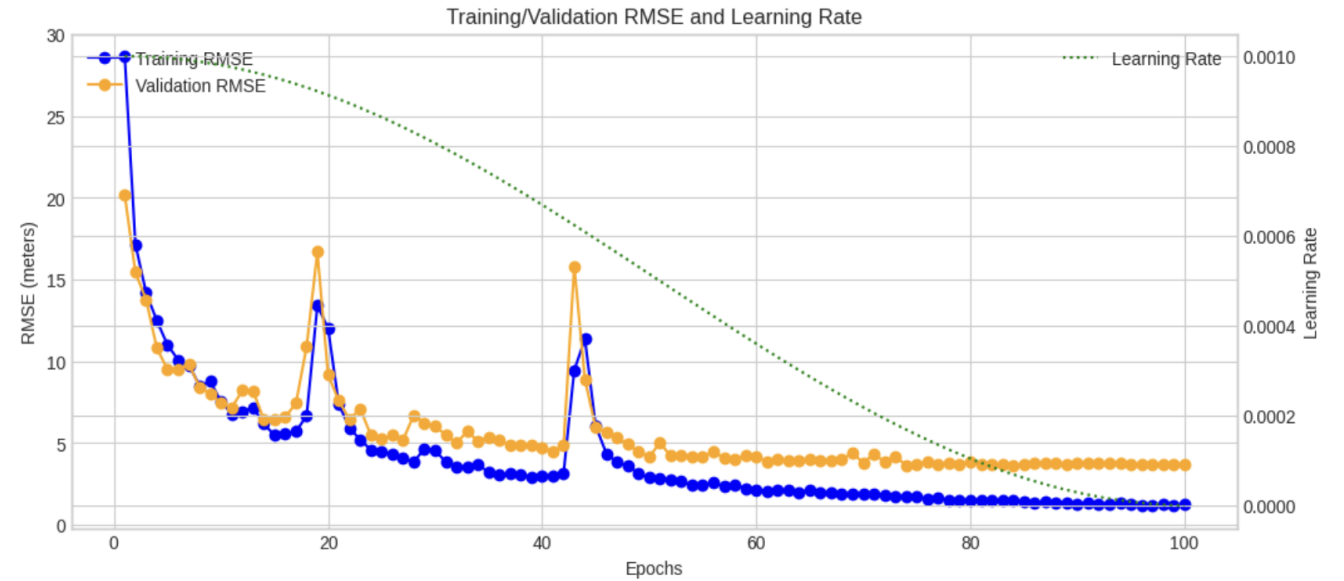
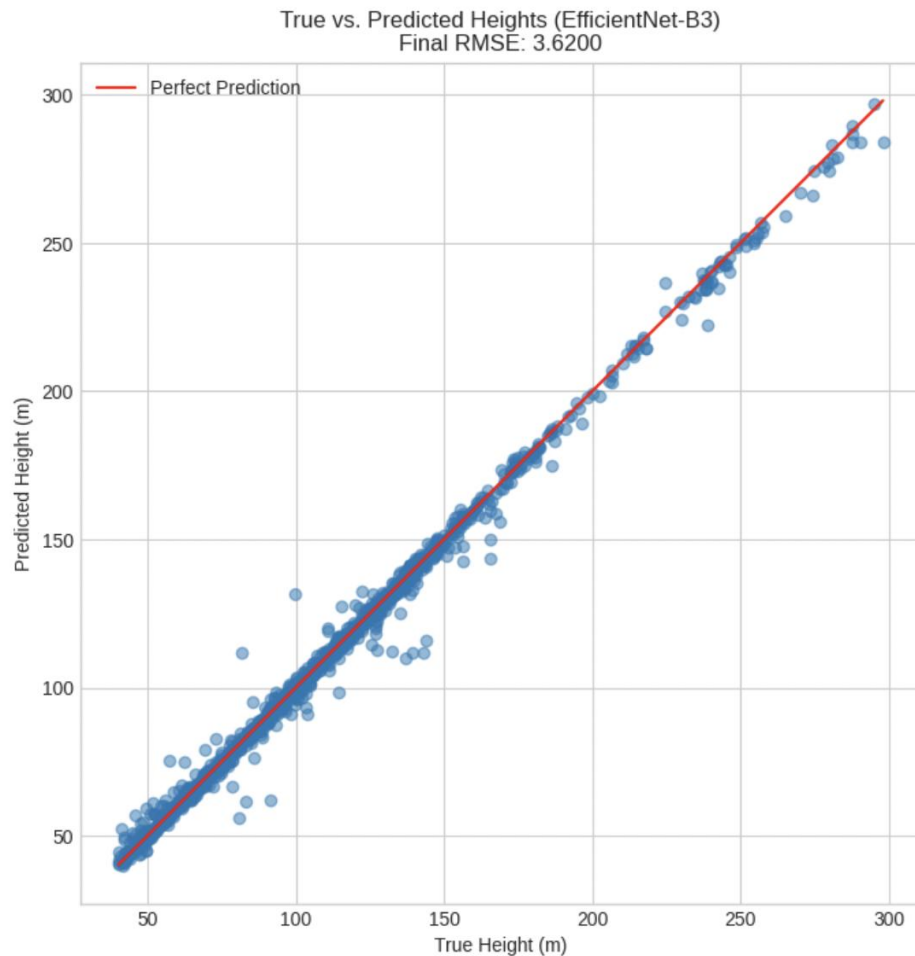
❖ 데이터 증강 코드

```
# 데이터 증강
IMG_SIZE = 224
train_transforms = A.Compose([
    A.Resize(height=IMG_SIZE, width=IMG_SIZE, interpolation=cv2.INTER_AREA),
    A.HorizontalFlip(p=0.5),
    A.Rotate(limit=20, p=0.7, border_mode=cv2.BORDER_CONSTANT, value=0),
    A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ToTensorV2()
])
val_transforms = A.Compose([
    A.Resize(height=IMG_SIZE, width=IMG_SIZE, interpolation=cv2.INTER_AREA),
    A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ToTensorV2()
])
```



3. 데이터 증강

❖ 3-5. 방법1) 가중 샘플링의 적용 결과 시각화



- Epoch : 100
- Val RMSE : 3.6200



3. 데이터 증강

❖ 3-5. 방법2) 데이터 증강을 구간 별로 다르게 적용

✓ 계기: 데이터 분석을 통한 문제점 발견

- 예측 오차가 무작위로 발생하는 것이 아니라 **50-200m** 라는 특정 높이 구간에 집중되어 있다는 패턴을 발견함

✓ 이유: "구간을 나눠 공략" 전략

- **기본 증강 (0-50m, 200m 이상 구간):** 이 구간은 모델이 이미 잘 맞추고 있으므로, 성능을 안정적으로 유지하면서 과적합을 방지하는 것이 목표였음. 따라서 **좌우 반전, 밝기/대비 조절**과 같은 기본적인 증강 기법만을 적용함
- **강화 증강 (50-200m 구간):** 오차가 집중된 이 문제 구간에서는 모델을 훨씬 더 강하게 훈련시킬 필요가 있음
- **회전, 색상 변형, 탄성 변형** 등을 추가하여 모델이 다양한 각도, 조명, 형태의 변화에도 흔들리지 않고 굴뚝의 본질적인 특징을 학습하도록 유도함
- **이미지 일부 가리기** 기법은 이미지의 일부를 무작위로 가려, 모델이 굴뚝의 특정 부분에만 의존하지 않고 전체적인 정보를 바탕으로 높이를 추론하도록함

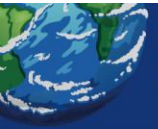


3. 데이터 증강

❖ 3-5. 방법2) 데이터 증강을 구간 별로 다르게 적용

✓ 기대효과

- **목표 구간 성능 향상:** 오차가 집중되었던 50-200m 구간의 RMSE가 집중적으로 감소하여, 모델의 전반적인 예측 정확도가 향상됨
- **모델 강건함 증가:** 강화된 증강 기법으로 훈련된 모델은 더 까다롭고 다양한 조건의 이미지에 대해서도 안정적인 예측 성능을 보일 것임
- **과적합 방지:** Regularization 효과가 있는 증강 기법을 통해 모델이 훈련 데이터에만 과도하게 최적화되는 것을 막고, 새로운 데이터에 대한 일반화 성능을 높일 수 있음



3. 데이터 증강

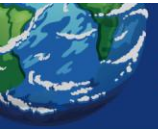
❖ 3-5. 방법2) 데이터 증강을 구간 별로 다르게 적용 코드

❖ 기본 증강(0-50m, 200m 이상 구간)

```
# 1. 기본 증강 (0-50m, 200m 이상 구간에 적용)
base_transforms = A.Compose([
    A.Resize(height=IMG_SIZE, width=IMG_SIZE, interpolation=cv2.INTER_AREA),
    A.HorizontalFlip(p=0.5),
    A.RandomBrightnessContrast(brightness_limit=0.2, contrast_limit=0.2, p=0.5),
    A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ToTensorV2()
])
```

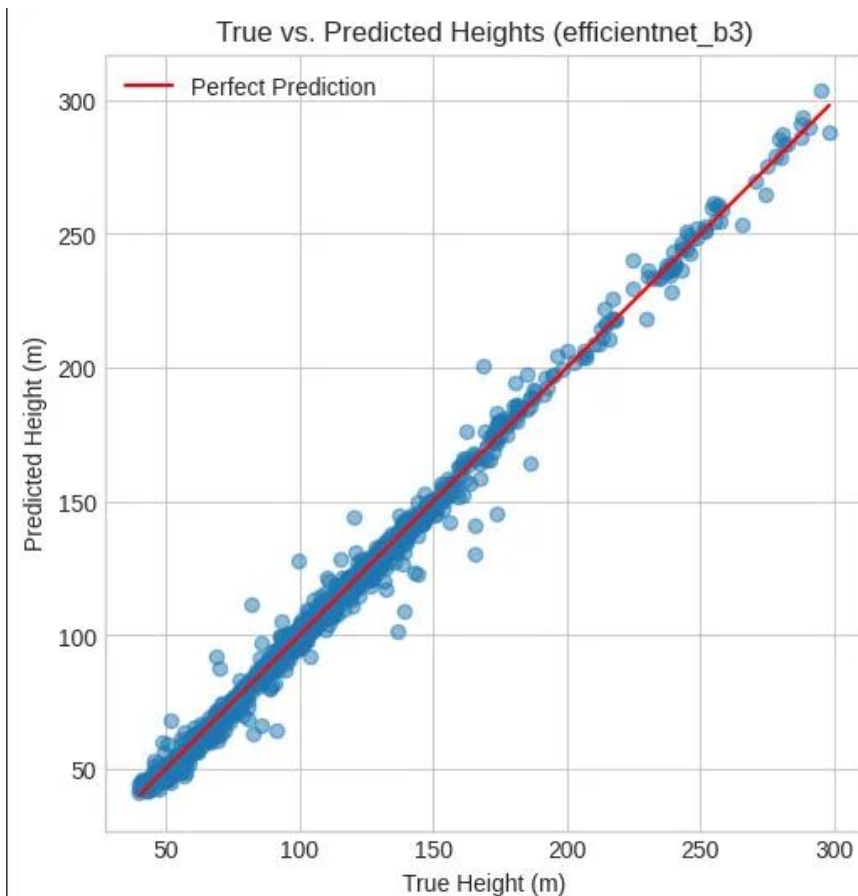
❖ 강화 증강(오차가 집중된 50-200m 구간에 적용)

```
# 2. 강화 증강 (오차가 집중된 50-200m 구간에 적용)
strong_transforms = A.Compose([
    A.Resize(height=IMG_SIZE, width=IMG_SIZE, interpolation=cv2.INTER_AREA),
    A.HorizontalFlip(p=0.5),
    A.Rotate(limit=30, p=0.5),
    A.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1, p=0.7),
    A.GaussianBlur(blur_limit=(3, 7), p=0.3),
    A.CoarseDropout(max_holes=8, max_height=25, max_width=25, p=0.3),
    A.ElasticTransform(p=0.2, alpha=120, sigma=120 * 0.05, alpha_affine=120 * 0.0
3),
    A.RandomBrightnessContrast(brightness_limit=0.2, contrast_limit=0.2, p=0.5),
    A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ToTensorV2()
])
```

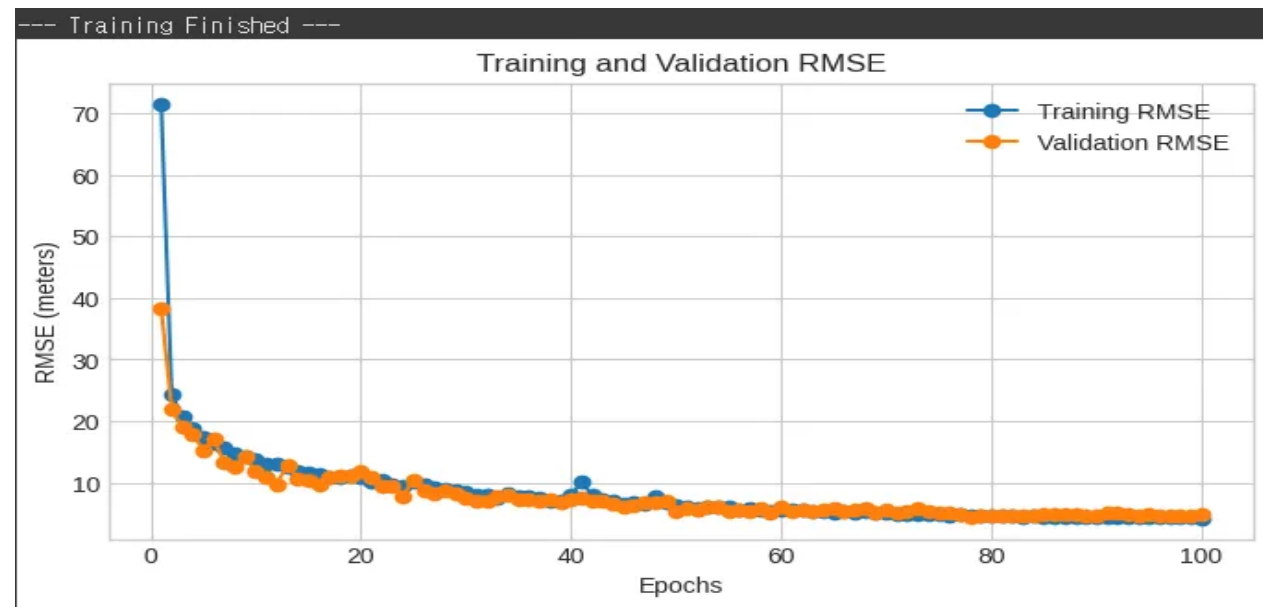


3. 데이터 증강

❖ 3-5. 방법2) 데이터 증강을 구간 별로 다르게 적용 결과 시각화



Final Best Validation RMSE: 4.4276



- Epoch : 100
- Val RMSE : 4.4276

결론: 더 좋은 성능을 가진 방법1)을 선택함!



4. 모델 설계

- ❖ 프로젝트 초기에는 이미지 분류 및 회귀 문제의 표준 모델인 **ResNet-50**으로 실험을 시작했으나, 최종적으로는 **EfficientNet-B3** 모델을 채택함

- ✓ **EfficientNet-B3 채택 이유 :**

- **효율성 및 성능 :** EfficientNet은 ResNet 계열 모델에 비해 적은 파라미터 수와 계산량으로 더 높은 성능을 내도록 설계된 최신 아키텍처
- **정확도 :** 실제 학습 결과, EfficientNet-B3 모델이 ResNet-50보다 더 낮은 RMSE 값을 기록하며 굴뚝 높이를 더 정밀하게 예측함. 따라서 더 효율적이면서도 뛰어난 성능을 보이는 EfficientNet-B3를 최종 모델로 선정함



5. 학습 단계

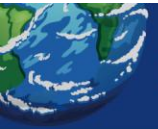
❖ 최적의 성능을 내기 위해 다음과 같은 하이퍼파라미터를 설정하여 학습을 진행

- **Imgsz = 224**: 입력 이미지의 크기를 224x224으로 조정하여 학습
- **Batch = 64**: 한 번의 반복에서 64개의 이미지를 사용하여 모델을 업데이트
- **Epoch = 100**: 전체 데이터셋을 총 100번 반복하여 학습
- **Learning Rate = 1e-3**: 모델이 가중치를 얼마나 빠르게 갱신할지를 결정하는 학습률
- **Weight Decay = 1e-3**: 과적합을 방지하기 위해 가중치 값이 과도하게 커지는 것을 억제함
- **Seed = 42**: 결과의 재현성을 위해 랜덤 시드를 42로 고정
- **Workers = 2**: 2개의 CPU 코어를 데이터 로딩에 사용하여 GPU의 대기시간을 최소화하고 학습 효율을 높임



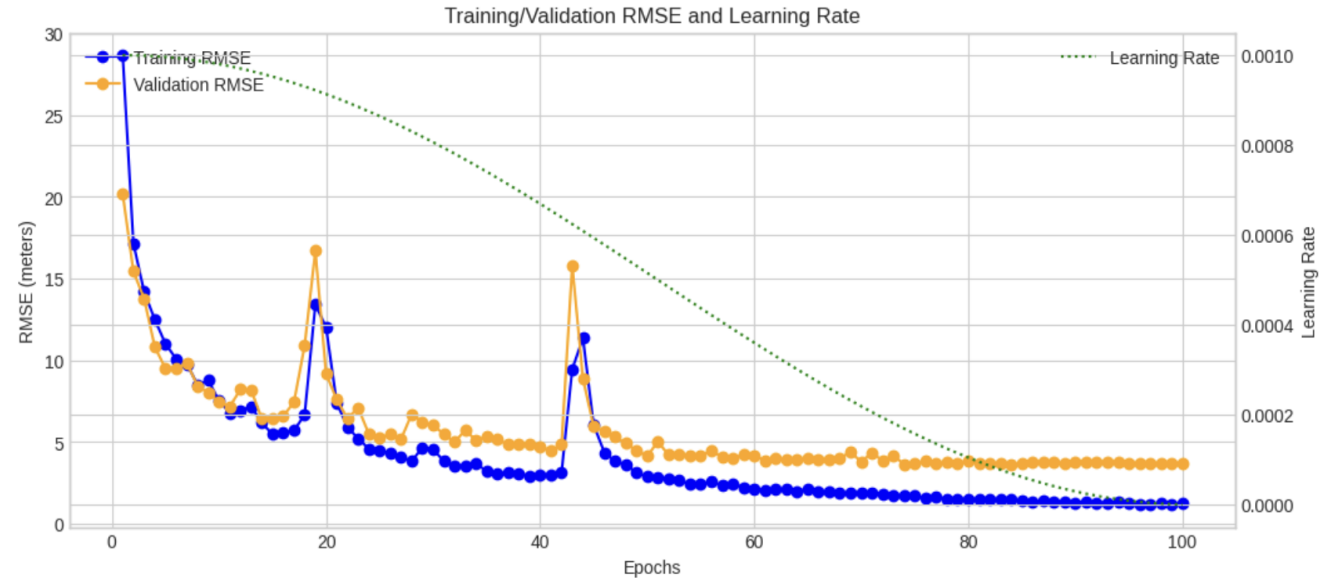
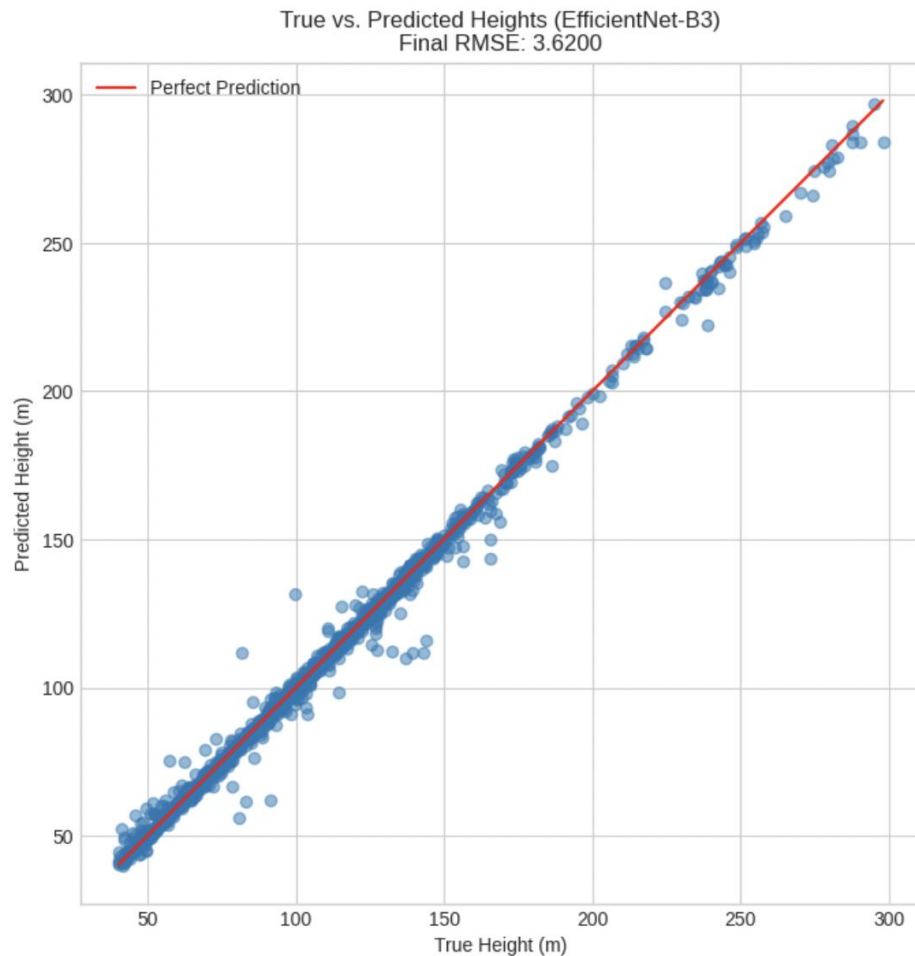
6. 추가 사항

- GPU: NVIDIA L4 GPU를 사용하여 학습을 진행함
- **Weighted Random Sampler** : 학습 데이터 중 예측이 특히 어려운 50-200m 높이 구간의 데이터에 1.5배의 가중치를 부여함. 이를 통해 모델이 학습 과정에서 해당 구간의 데이터를 더 자주 접하게 하여, 특정 높이 구간에서의 예측 성능을 집중적으로 개선함
- 옵티마이저 및 스케줄러 :
 - **AdamW** : 기본적인 Adam Optimizer에 Weight Decay를 개선한 AdamW를 사용하여 안정적인 학습을 도모함
 - **CosineAnnealingLR** : Learning Rate을 코사인 함수 형태에 따라 점진적으로 감소시키는 코사인 어닐링 스케줄러를 적용했음. 이는 학습 후반부에 모델이 최적점에 더 잘 수렴하도록 돕는 효과적인 기법임



7. 수행 결과

❖ 100 epoch 학습 완료 후, 검증 데이터셋에 대한 최종 성능은 매우 성공적임



- Epoch : 100
- Val RMSE : 3.6200



7. 수행 결과

➤ 결과 해석 :

- 최종 검증 RMSE가 약 3.62m로, 이는 모델의 높이 예측값과 실제 값의 평균 오차가 약 3.62m임을 의미함. 위성 이미지만으로 굴뚝의 높이를 이 정도의 오차 내에서 예측한 것은 매우 높은 정확도를 달성했음을 보여줌
- RMSE 학습 곡선 : 학습 초반에 검증 RMSE가 빠르게 감소하다가 점차 안정화되는 양상을 보임. 코사인 어닐링 스케줄러의 영향으로 학습률이 점차 줄어들면서 후반부에도 꾸준히 성능이 개선되는 것을 확인할 수 있었음
- Scatter Plot : 실제 높이와 예측 높이를 산점도로 시각화한 결과, 데이터 포인트들이 $y = x$ 직선 주위에 매우 밀집해 있는 것을 확인함. 이는 모델이 특정 높이에 치우치지 않고 전체 높이 구간에 걸쳐 일관되고 정확한 예측을 하고 있음을 시각적으로 증명함



Mission 3: 산업단지 영역 분할



1. 데이터 이해

✓ 목표

Sentinel-2 위성 이미지에서 **산업단지 픽셀을 분할**(Semantic Segmentation)

✓ 데이터 소스

Sentinel-2 산업 단지 이미지 데이터

✓ 데이터 구성

- **학습 데이터** : 산업단지 이미지/라벨링 8,000개 (TS_SN10_SN10 폴더)
- **검증 데이터** : 산업단지 이미지/라벨링 1,000개 (VS_SN10_SN10 폴더)



1. 데이터 이해

✓ 파일 형식/ 메타데이터 이해

driver : GeoTIFF (Gtiff)

dtype : uint16 -> 데이터 타입은 16비트 정수 (0~65535 범위)

nodata : none -> 누락값이 따로 지정되지 않음

width x height : 512 x 512 픽셀 -> 영상의 가로, 세로 픽셀 수

count : 4 -> 총 4개의 밴드(채널)이 들어있음

crs : None -> 좌표계(CRS)가 지정되어 있지 않음



2. 데이터 처리

❖ 원천 위성 이미지(.tif) + 라벨(.tif) 쌍을 PyTorch 학습에 바로 쓸 수 있게 전처리 진행

✓ 파일 형식 변환:

원천 위성 이미지(.tif) + 라벨(.tif) 쌍을 .pt파일로 변환하여 PyTorch 학습에 바로 쓸 수 있게 진행

✓ 이미지/레이블 짝 맞추기:

img_names.intersection(label_names)로 동일한 이름의 tif 파일만 선택 -> 데이터 누락 방지

✓ 데이터 읽기(입력/라벨):

라이브러리 : rasterio

이미지 : sr.read() -> (C,H,W) = (4,512,512)

라벨 : sr.read(1) -> (H,W)



2. 데이터 처리

❖ 원천 위성 이미지(.tif) + 라벨(.tif) 쌍을 PyTorch 학습에 바로 쓸 수 있게 전처리 진행

✓ 정규화(입력 스케일):

- `image_tensor = torch.from_numpy(image_array).float() / 65535.0`
 - 16비트 영상을 [0,1] 범위로 변환

✓ 마스크 클래스 선택:

- `label_array = (label_array == 10).astype(np.int64)`
- 특정 클래스 값(10)을 binary mask로 변환(산업단지만 1, 나머지 0)
 - 이진 분할(Binary segmentation)에 맞춤



2. 데이터 처리

❖ 원천 위성 이미지(.tif) + 라벨(.tif) 쌍을 PyTorch 학습에 바로 쓸 수 있게 전처리 진행

✓ .pt 파일 저장 예시:

```
{  
    "image": torch.Size([4,512,512]) #4채널, 512 x 512 크기  
    "label": torch.Size([512,512])   #1채널, 이진 마스크  
}
```



3. 데이터 증강

✓ 크기/이동/회전 (장면 규모 다양성 확대)

➤ Shift/Scale/Rotate

- **shift_limit**: 0.06 (유지)
- **scale_limit**: $\pm 0.10 \rightarrow \pm 0.50$ → 센서/고도/해상도 차이에 대응, 소형·대형 구조 모두 학습
- **rotate_limit**: $15^\circ \rightarrow 20^\circ$ → 경계 왜곡 없이 약간 더 다양한 자세 노출

➤ 적용 확률: $p = 0.7$

✓ 비선형 왜곡 (현실성 유지 수준으로 완화)

➤ GridDistortion: **distort_limit 0.10, $p = 0.3$**

- 잔잔한 격자 왜곡만 허용, 비현실적 휨 방지

➤ OpticalDistortion: **distort_limit 0.10, shift_limit 0.05, $p = 0.2$**

- 광학 왜곡을 "약하게"만 부여하여 구조 보존



3. 데이터 증강

- ✓ 방향/회전 (invariance 확보, 원본 분포도 보전)
 - 수평 뒤집기: $p = 0.5$ (유지)
 - 블록/도로 패턴에 유익, 과하지 않음
 - 수직 뒤집기: $p = 0.5 \rightarrow 0.3$
 - 위성 장면의 상·하 문맥(그림자·태양방위) 보존을 위해 과도한 반전 빈도 완화
 - 90도 회전: $p = 0.5$ (유지)
- ✓ 광도 변형 (스펙트럼 보존 중심)
 - RandomBrightnessContrast: $\pm 0.20 \rightarrow \pm 0.08$, $p = 0.4$
 - 4채널 위성의 분광 특성 훼손 최소화, 과도한 명암 변동 억제
- ✓ 가림/노이즈 (과적합 방지, 과한 손상은 제한)
 - CoarseDropout: max_holes 6, 32×32 , $p = 0.4$
 - 소규모 occlusion만 추가, 핵심 경계 학습 방해하지 않음



4. 모델 설계

❖ 모델 기본 구조: ResNet34 + smp.Unet

- ✓ **U-Net (전체 구조):** 이미지를 압축하여 특징을 추출하는 Encoder와, 추출된 특징을 다시 원래 이미지 크기로 복원하여 분할 맵을 만드는 Decoder로 구성
 - **Encoder: ResNet34**
 - ImageNet으로 사전 학습된(Pre-trained) 모델을 사용하여 전이 학습(Transfer Learning)
 - 입력 이미지로부터 강력하고 의미있는 특징(Feature) 추출 담당
 - **Skip Connection:** Encoder의 공간 정보(위치)와 Decoder의 의미 정보(특징)를 결합하여 정교한 분할 맵 생성
 - **smp.Unet**
 - segmentation-models-pytorch 라이브러리를 사용하여 복잡한 U-Net 모델 아키텍처를 직접 코드로 구현할 필요 없이 진행



4. 모델 설계

❖ 손실 함수 = CE Loss + Dice Loss

✓ CrossEntropy()

- 이미지의 모든 픽셀을 개별적으로 분류하는 문제(Pixel-wise Classification)로 볼 수 있기 때문에 CE 손실 함수를 사용

✓ DiceLoss()

- 이미지 분할(Segmentation) 에서 널리 사용되는 **Dice Coefficient**라는 유사도 측정 지표를 손실 함수로 변환하여 사용



5. 학습 단계

❖ 최적의 성능을 내기 위해 다음과 같은 하이퍼파라미터를 설정하여 학습을 진행

- **Imgsz = 512**: 원본 타일 크기(512×512)를 그대로 사용해 학습
- **Batch = 32**: 한 번의 반복에서 32개의 이미지를 사용하여 모델을 업데이트
- **Epoch = 80**: 전체 데이터셋을 총 80번 반복하여 학습
- **Seed = 42**: 결과의 재현성을 위해 랜덤 시드를 42로 고정
- **Num_workers = 2**: 데이터 로딩에 2개의 CPU 워커를 사용하여 GPU 대기 시간 감소 및 학습



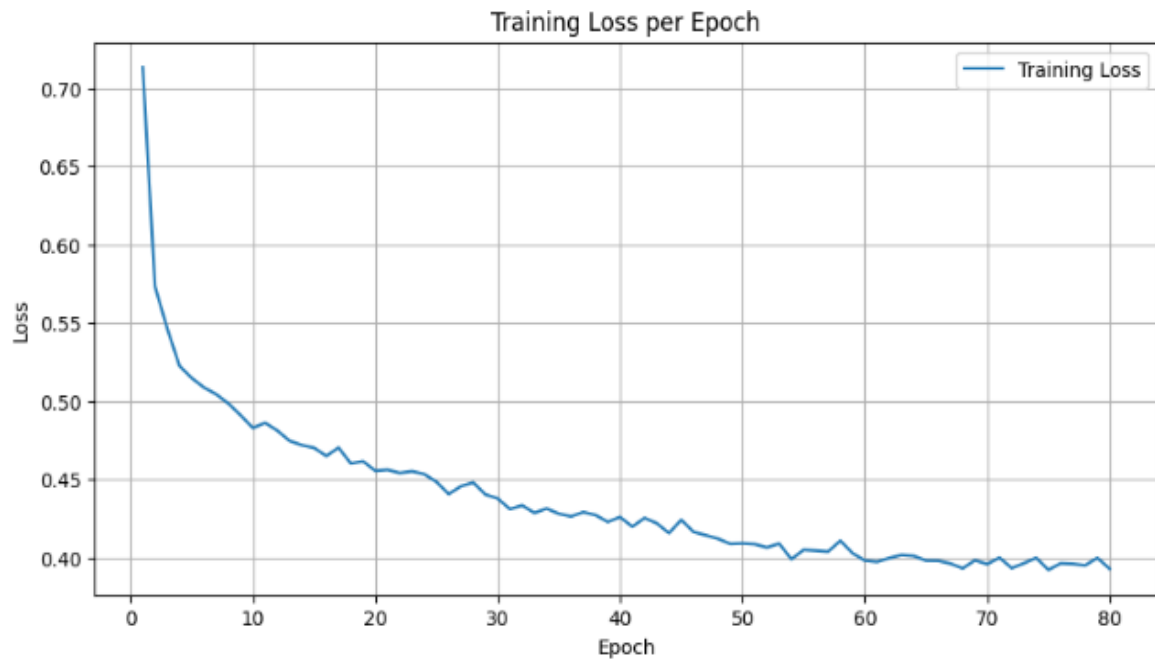
6. 추가사항

- GPU : NVIDIA A100 GPU를 사용하여 대규모 이미지 데이터를 빠르고 효율적으로 학습
- 하이퍼파라미터 최적화 (Optimization)
 - 옵티마이저 (Optimizer): AdamW
 - 학습률 스케줄러 (LR Scheduler): CosineAnnealingLR

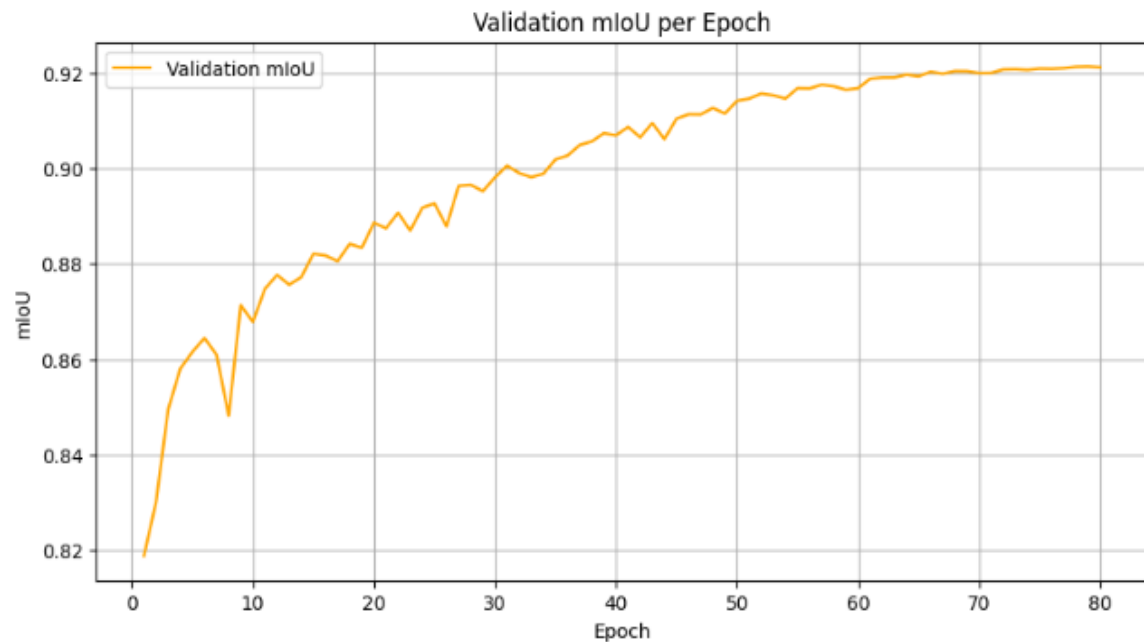


7. 수행 결과

❖ Training Loss



❖ Validation mIoU





7. 수행 결과

❖ 산업단지 영역 분할 결과 샘플

Ground Truth



Model Prediction



Ground Truth



Model Prediction





7. 수행 결과

❖ 결과 분석

- Training Loss: 0.3898
 - 학습이 진행됨에 따라 손실 값이 꾸준히 감소하여 약 0.4 이하로 안정적으로 수렴
 - 그래프에 큰 변동이나 갑작스러운 급등이 없어 학습이 안정적으로 진행됨
- Validation mIoU: 0.9211
 - 최종 mIoU가 약 0.92까지 상승한 것은 모델이 처음 보는 데이터에 대해서도 매우 높은 예측 성능을 보인다는 것을 의미
 - 학습 손실이 감소함에 따라 검증 mIoU도 꾸준히 상승하는 것은 모델이 과적합(Overfitting) 되지 않고 잘 일반화 된 것을 보여줌
- 산업단지 분할 결과
 - 크고 명확한 산업단지 형태는 Ground Truth와 매우 유사하게 잘 예측



ESG 시사점: E (Environment | 환경)

- 대기오염원 정밀 관리 및 배출량 추정
 - 산업단지 영역 분할 기술로 모니터링할 지역 대상 특정
 - 특정 산업단지에서 발생하는 대기오염물질의 총량을 더 정확하게 추산하고 관리
- 미신고/불법 배출원 식별 및 감시
 - 굴뚝 탐지를 통해 환경 규제 사각지대를 해소 및 감독 기관의 단속 효율성 극대화
 - 환경오염을 사전 예방
- 산업단지의 환경 영향 총량 관리
 - 굴뚝 탐지를 통해 단지 내 오염원의 밀도 변화를 지속적으로 추적
 - 신규 산업단지 조성이나 기존 단지 확장에 따른 환경 영향 사전 평가
 - 지속가능한 개발 계획을 수립 가능



ESG 시사점: S (Social | 사회)

- 지역 사회의 건강 및 안전 증진
 - 어떤 지역 사회가 산업 오염원에 더 많이 노출되어 있는지 객관적으로 파악
 - 환경오염에 취약한 계층이나 지역을 식별하고, 이들에 대한 건강영향평가나 정책적 지원의 우선순위를 정하는 데 활용
- 정보 접근성 향상: 공개 대시보드로 지역사회가 변화 정보를 쉽게 확인
 - 기업은 위성 및 AI 분석을 통한 객관적인 데이터를 활용하여 자사의 환경 관리 노력을 외부에 투명하게 공개 가능



ESG 시사점: G (Governance | 거버넌스)

➤ 데이터 기반 환경 규제 및 정책 수립

- 원격 탐사 및 AI 분석을 통한 전국적인 산업단지와 오염원 모니터링하는 시스템 구축
- 점검기록(모델 버전, 데이터, 판단 근거) 감사로그로 정책 집행 투명성 강화

➤ ESG 경영 성과 측정 및 외부 검증

- 기업의 '그린워싱(Greenwashing)' 방지
- ESG 정보의 신뢰도 향상

2025 데이터 크리에이터 캠프

감사합니다

